

Statistical testing in latent variable models

Julia Świątkowska

Statistical testing in latent variable models is crucial for distinguishing between true signals and noise. Without proper statistical controls, the results of latent variable analyses can be misleading, particularly in high-dimensional datasets where the risk of false discoveries is high. For example, in genomics, identifying genes associated with latent factors like cell-cycle regulation or disease subtypes requires rigorous statistical methods to ensure that the identified associations are not merely artifacts of overfitting.

1. Methods

1.1 Boruta

Unlike traditional feature selection methods that focus on finding a minimal set of features for optimal predictive performance (the minimal-optimal problem), Boruta aims to solve the all-relevant problem. This is particularly important in fields like genomics, where understanding the full set of genes or variables associated with a biological process is more valuable than simply building a predictive model. The Boruta algorithm (implemented in R) is wrapped around the random forest classification - an ensemble method that builds multiple decision trees on bootstrapped samples of the data. However, Random Forest importance scores can be influenced by random noise and correlations, making it difficult to determine statistical significance. Boruta overcomes this by introducing random probes—shadow features created by shuffling the values of real features. This helps Boruta distinguish truly important features from irrelevant ones.

How does the Boruta algorithm work? The most important steps:

1. Extend the dataset by adding copies of all variables (shadow features are added)
2. Run the random forest on the extended information, gather the Z (importance) scores
3. Real features are compared to the maximum importance of shadow features. Features consistently outperforming shadows are marked as relevant; others are removed
4. Repeat the procedure until the importance is assigned for all the attributes, or the algorithm has reached the previously set limit of the random forest runs

By using random probes, it ensures statistically sound identification of all relevant variables, making it invaluable for uncovering meaningful patterns in complex data [1].

1.2 Jackstraw

The Jackstraw method is a resampling technique that helps find statistically significant variables associated with latent factors in high-dimensional data, especially in Principal Component Analysis (PCA). PCA is useful for identifying patterns in data, but it can be difficult to tell which specific variables (like genes) are truly responsible for these patterns due to overfitting. Jackstraw solves this problem by introducing synthetic null variables—shuffled versions of the real ones—to estimate what random associations would look like. This makes it easier to test which variables are genuinely significant.

Jackstraw workflow:

1. Perform PCA on the original dataset to obtain the principal components (PCs)
2. Replace a small subset of variables with permuted versions, preserving the overall data structure
3. Perform PCA on the modified dataset and calculate association statistics (e.g., F-statistics) between the synthetic null variables and the PCs
4. Compare the association statistics of the original variables to those of the synthetic nulls to compute p-values, adjusting for overfitting
5. Repeat the process to ensure robust estimation of the null distribution

Jackstraw has been used to identify key genes in yeast and post-trauma patients. While selecting the right number of PCs is crucial, its overfitting control makes it a strong tool for testing latent variable models [2].

1.3 Bootstrap

The Bootstrap method is used to estimate the uncertainty associated with an estimator or statistical learning method. The power of the bootstrap lies in the fact that it can be easily applied to a wide range of statistical learning methods, including some for which a measure of variability is otherwise difficult to obtain and is not automatically output by statistical software.

How the Bootstrap works:

1. From the original dataset, generate multiple bootstrap samples by randomly selecting observations with replacement. This means that some observations may appear multiple times in a bootstrap sample, while others may not appear at all
2. For each bootstrap sample, compute the statistic of interest (e.g., a regression coefficient or an investment allocation parameter α)
3. Use the distribution of the bootstrap estimates to calculate the standard error or confidence intervals for the statistic. For example, the standard error of α can be estimated using the formula (where B - nr. of bootstrap samples, $\hat{\alpha}^{*r}$ - estimate from the r -th bootstrap sample:

$$SE_B(\hat{\alpha}) = \sqrt{\frac{1}{B-1} \sum_{r=1}^B \left(\hat{\alpha}^{*r} - \frac{1}{B} \sum_{r'=1}^B \hat{\alpha}^{*r'} \right)^2}.$$

In the example, the bootstrap estimates the optimal investment allocation α that minimizes risk. By generating bootstrap samples, it produces a distribution of α estimates, allowing researchers to quantify uncertainty [3].

References

1. Miron B. Kursu, Witold R. Rudnicki, Feature Selection with the Boruta Package, *Journal of Statistical Software*, Volume 36, Issue 11, September 2010,
2. Neo Christopher Chung, John D. Storey, Statistical significance of variables driving systematic variation in high-dimensional data, *Bioinformatics*, Volume 31, Issue 4, February 2015, Pages 545–554,
3. Gareth James, Daniela Witten, Trevor Hastie, Robert Tibshirani, Jonathan Taylor, An Introduction to Statistical Learning with Applications in Python, July 5, 2023